Single-source shortest paths: solutions

24-E01 In-place and synchronous states differ

Derivation

In(s, a, b, c, t, z) order, start with(0,∞,∞,∞,∞,∞). Round 1 uses sa to set a4(parent s), then sb to set b1(parent s); other edges lack finite sources. Round 2 uses at to set t12(parent a), ac to set c5(parent a), then ba to set a−1(parent b). Round 3 first sets t8 via ct(parent c), then t7 via at(parent a), then c0 via ac(parent a). Round 4 sets t3 via ct(parent c). Round 5 has no update.

Round-end vectors are(0,4,1,∞,∞,∞),(0,−1,1,5,12,∞),(0,−1,1,0,7,∞),(0,−1,1,0,3,∞), followed by the same vector. Final parents give s→b→a→c→t.

With order sa, sb, ba, ac, ct, at, bc, bt, round 1 immediately reuses b1 to get a−1, c0, t3 and reaches the final values. Synchronous DP reads only D_0 in its first round and obtains only a4, b1. Both scan all edges once, but use different old/new state dependencies.

Conclusion and checks

An in-place round captures at least the corresponding edge-count paths, not only those paths. Synchronous D_i means exactly the optimum over at most i edges. The specified original order first permits early stopping in round 5.

24-E02 Negative-cycle certificates and affected vertices

Derivation

Cycle a→c→a costs 1−2=−1 and is entered along s→b→a. Walk s→b→a→(c→a repeated r times)→c→t costs 1−2−r+1+3=3−r, unbounded below. Thus a, c, t have distance−∞. No cycle exit returns to s, whose distance is 0. Vertex b has only direct entry from s and distance 1. Vertices z, p, q are unreachable. The eight entries are(0,−∞,1,−∞,−∞,∞,∞,∞).

Let $B=\{v:D_n[v]<D_{n-1}[v]\}$. If a seed's optimal n-edge walk had only nonnegative removable cycles, deleting them would leave an at-most-(n−1)-edge path of no greater cost, a contradiction. Every seed is therefore affected by a negative cycle, as are its forward descendants. Conversely, all vertices of a reachable negative cycle have finite D_{n-1}. If none were seeds, its edges would satisfy $D_{n-1}[v]\leq D_{n-1}[u]+w$; summing gives $0\leq$a negative total. Every such cycle has a seed, so forward search covers all affected vertices.

Single-source shortest paths: solutions (continued)

Conclusion and checks

The negative pq cycle has no entry from s. Detecting any negative cycle anywhere would require a separate super-source problem. Mark descendants reachable from cycles, not every ancestor capable of entering them.

24-E03 Shortest paths and critical paths

Derivation

Order s, b, a, c, t, z is topological: every arc goes forward. Scanning s gives a4, b1; b gives a−1, c6, t13; a gives c0, t7; c gives t3. Vertices t, z have no outgoing edges. The result is$(0,-1,1,0,3,\infty)$. Negative ba is harmless because no directed cycle exists.

Job starts are S[A]=S[B]=0, with F[A]=3, F[B]=2. Then S[C]=max(3,2)=3, F[C]=7, S[D]=7, F[D]=8. Critical path A, C, D costs 3+4+1=8. Total work is 3+2+4+1=10.

Conclusion and checks

Unlimited processors start A, B together and attain 8. One processor must perform10 units of work, so cannot finish at 8. Schedule A[0,3], B[3,5], C[5,9], D[9,10] attains 10. Dependency and resource lower bounds are distinct.

24-E04 Dijkstra's no-reopening contract

Derivation

The nonnegative version extracts s0 with candidates a4, b1. Extract b1 to obtain a3(parent b), c6(parent b), t13(parent b). Extract a3 to obtain c4(parent a), t11(parent a). Extract c4 to obtain t7(parent c). Extract t7, then stop with z∞. Finalized values 0,1,3,4,7 form tree s-b-a-c-t. Discovery of t13 leaves smaller candidates capable of improving it; extraction at 7 establishes finality.

In the counterexample, after s0 come a2, b5. Extract a2 and assign t4; then extract t4 before b5. Edge ba−4 would make a1 and then t3, but both are finalized. On the true shortest s-b-a path, intermediate b has distance 5 greater than a's distance 1. The implication from nonnegative suffix to $\delta(s, b) \leq \delta(s, a)$ fails.

Conclusion and checks

The counterexample is a DAG and has no negative cycle. A violated nonnegative-weight contract causes failure. DAG shortest paths instead gives a1, t3 in linear time.

Single-source shortest paths: solutions (continued)

24-E05 Two certificates for difference constraints

Derivation

Arcs are 1→2:−2,2→3:−1,3→1:4. An all-zero scan gives x=(0,−2,−3), which no later round changes. Differences are−2,−1,3, within bounds−2,−1,4. Add 3 to obtain(3,1,0), preserving every difference and setting the minimum coordinate to 0.

With final bound 2, summing $(x_2 - x_1) + (x_3 - x_2) + (x_1 - x_3) = 0$ gives 0≤−2−1+2=−1, proving no solution exists. Equality $x_1 - x_3 = 3$ adds 3→1:3 and 1→3:−3. The original feasible vector satisfies it.

For integer variables, floor the real bounds to−2,−1,2. Integer differences make this equivalent to the stated integer system, which has the same negative-cycle contradiction and is infeasible. The real-valued system is feasible, for example at(0,−1.2,−1.4), whose differences−1.2,−0.2,1.4 satisfy the original real bounds. Integrality changes feasibility.

Conclusion and checks

The original system is feasible and translation invariant. The changed final bound 2 and the all-integer variant are infeasible. Arcs go from the subtracted variable toward the positively signed variable.

24-E06 Zero-weight predecessors and independent verification

Derivation

Parents a←b, b←a form a cycle unreachable from s in the parent-children graph. Each parent is tight but realizes no s-to-a or s-to-b path. Valid parents are parent[a]=s, parent[b]=a, with all distances0.

A tight rooted tree realizes a path of cost d[v], so δ(s, v)≤d[v]. Input inequalities summed along any walk give d[v]≤every walk cost and therefore d[v]≤δ(s, v). Together they establish equality.

Checking only inequalities would accept d[s]=0, d[a]=−100 for the sole edge sa1, since−100≤1, despite no realizing path. Tight-root checks reject it. Checking only a tight rooted tree would accept sa5, sb1, ba1 with parents s for both a, b and d=(0,5,1). But ba violates5≤1+1, and true distance to a is 2.

Single-source shortest paths: solutions (continued)

Conclusion and checks

Equal distances on zero edges do not ensure acyclic predecessor directions. Relaxation histories use update times to prove acyclicity; external results must explicitly verify rooted connectivity.

24-E07 Reconstructing a cycle from the edge-count DP

Derivation

In(s, a, b) order, $D_0=(0,\infty,\infty)$, $D_1=(0,1,\infty)$, $D_2=(0,1,-2)$, $D_3=(0,-1,-2)$. Vertex a strictly improves in layer 3. Backtracking $a_3\leftarrow b_2\leftarrow a_1\leftarrow s_0$ gives $s\rightarrow a\rightarrow b\rightarrow a$, cost $1-3+1=-1$. Repeated a extracts cycle $a\rightarrow b\rightarrow a$, weight−2, reachable from s.

In general a walk with fewer than n edges would already qualify in layer n−1, contradicting strict improvement. An n-edge walk repeats a vertex but can contain positive, zero, and negative cycles. Decomposing it must reveal a negative one: if all were nonnegative, deleting all cycles would give a simple at-most-(n−1)-edge path no more expensive, another contradiction. A repeated vertex alone says nothing about cycle sign.

Conclusion and checks

A full n-layer by n-vertex predecessor table needs $O(n^2)$ auxiliary space plus O(m) input. Two distance layers use O(n) but omit this reconstruction history. Saving space while retaining certificates needs an additional algorithm and proof.

24-E08 Algorithm choices and necessary cost assumptions

Derivation

(a) BFS takes Θ(n+m), with edge count as cost. (b) Topological DAG shortest paths takes Θ(n+m) and permits negative edges. (c) Dijkstra finalizes extractions under nonnegative weights. (d) Bellman–Ford detects reachable negative cycles; complete distance semantics additionally search forward from layer-n seeds and mark−∞, rather than outputting decreasing finite estimates as answers.

At the given n, m, Bellman–Ford's leading nm is 3×10^8 edge scans. Binary Dijkstra has$(n+m)\log_2 n\approx4\times10^4\times13.288\approx5.32\times10^5$ in operation order. Fibonacci has $m+n\log_2 n\approx1.63\times10^5$ amortized order. Linear-key scanning has $n^2+m\approx1.0003\times10^8$. Basic operations and constants differ, so these are not measured time ratios.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Single-source shortest paths: solutions (continued)

Conclusion and checks

Even m=0 requires initializing and returning n entries, an Ω(n) cost. Bellman–Ford and all-zero difference constraints use O(n+nm). An explicit super-source adds n arcs, so original m=0 does not mean the augmented graph has no edges.

Verification code appendix: dijkstra_paths.py

```
"""Dijkstra on a nonnegative weighted graph."""

from heapq import heappop, heappush

def dijkstra(graph, source):
    distance = {v: float("inf") for v in graph}
    distance[source] = 0
    queue = [(0, source)]
    while queue:
        current, u = heappop(queue)
        if current != distance[u]:
            continue
        for v, weight in graph[u].items():
            candidate = current + weight
            if candidate < distance[v]:
                distance[v] = candidate
                heappush(queue, (candidate, v))
    return distance


graph = {"s": {"a": 2, "b": 5}, "a": {"b": 1}, "b": {}}
assert dijkstra(graph, "s") == {"s": 0, "a": 2, "b": 3}
print(dijkstra(graph, "s"))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Single-source shortest paths: solutions (continued)

Verification code appendix: shortest_path_checks.py

```
"""Check Bellman-Ford, DAG relaxation, Dijkstra, and difference constraints."""

from math import inf
import heapq
import random


def bellman_ford(vertices, edges, source):
    distance = {vertex: inf for vertex in vertices}
    distance[source] = 0
    for _ in range(len(vertices) - 1):
        for u, v, weight in edges:
            if distance[u] != inf and distance[v] > distance[u] + weight:
                distance[v] = distance[u] + weight
    negative_cycle = any(
        distance[u] != inf and distance[v] > distance[u] + weight
        for u, v, weight in edges
    )
    return distance, negative_cycle


def dijkstra(vertices, adjacency, source):
    distance = {vertex: inf for vertex in vertices}
    distance[source] = 0
    queue = [(0, source)]
    visited = set()
    while queue:
        current, u = heapq.heappop(queue)
        if current != distance[u]:
            continue
        if u in visited:
            continue
        visited.add(u)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Single-source shortest paths: solutions (continued)

```
        for v, weight in adjacency.get(u, ()):
            candidate = current + weight
            if candidate < distance[v]:
                distance[v] = candidate
                heapq.heappush(queue, (candidate, v))
    return distance


def dag_shortest_paths(vertices, edges, source):
    adjacency = {vertex: [] for vertex in vertices}
    indegree = {vertex: 0 for vertex in vertices}
    for u, v, weight in edges:
        adjacency[u].append((v, weight))
        indegree[v] += 1
    ready = [vertex for vertex in vertices if indegree[vertex] == 0]
    order = []
    while ready:
        u = ready.pop()
        order.append(u)
        for v, _ in adjacency[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                ready.append(v)
    assert len(order) == len(vertices)
    distance = {vertex: inf for vertex in vertices}
    distance[source] = 0
    for u in order:
        if distance[u] == inf:
            continue
        for v, weight in adjacency[u]:
            distance[v] = min(distance[v], distance[u] + weight)
    return distance
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Single-source shortest paths: solutions (continued)

```
def run():
    sample_edges = [("s", "a", 4), ("s", "b", 5), ("a", "b", -3)]
    distance, has_negative_cycle = bellman_ford({"s", "a", "b"}, sample_edges, "s")
    assert distance == {"s": 0, "a": 4, "b": 1}
    assert not has_negative_cycle

    cycle_edges = [("s", "a", 1), ("a", "b", -3), ("b", "a", 1)]
    _, has_negative_cycle = bellman_ford({"s", "a", "b"}, cycle_edges, "s")
    assert has_negative_cycle

    dag = dag_shortest_paths(
        {"s", "a", "b"},
        [("s", "a", 3), ("s", "b", 8), ("a", "b", -2)],
        "s",
    )
    assert dag == {"s": 0, "a": 3, "b": 1}

    adjacency = {"s": [("a", 2), ("b", 5)], "a": [("b", 1)]}
    assert dijkstra({"s", "a", "b"}, adjacency, "s") == {"s": 0, "a": 2, "b": 3}

    difference_solution = {"a": 2, "b": 0, "s": 0}
    assert difference_solution["a"] - difference_solution["b"] <= 2
    assert difference_solution["b"] - difference_solution["s"] <= 5

    negative_edge = {
        "s": [("a", 2), ("b", 1)],
        "a": [("b", -2)],
        "b": [("c", 2)],
    }
    wrong = dijkstra({"s", "a", "b", "c"}, negative_edge, "s")
    assert wrong["c"] == 3
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Single-source shortest paths: solutions (continued)

```
    exact, _ = bellman_ford(
        {"s", "a", "b", "c"},
        [("s", "a", 2), ("s", "b", 1), ("a", "b", -2), ("b", "c", 2)],
        "s",
    )
    assert exact["c"] == 2

    rng = random.Random(24)
    cases = 0
    for size in range(2, 8):
        names = [f"v{i}" for i in range(size)]
        for _ in range(20):
            edges = []
            for i in range(size):
                for j in range(i + 1, size):
                    if rng.random() < 0.4:
                        edges.append((names[i], names[j], rng.randint(-5, 9)))
            bellman = bellman_ford(names, edges, names[0])[0]
            dag = dag_shortest_paths(names, edges, names[0])
            assert bellman == dag
            cases += 1

    print(
        f"PASS Bellman-Ford_negative_cycle=yes DAG_cases={cases} "
        "Dijkstra_negative_edge_failure=yes difference_constraints=yes"
    )


if __name__ == "__main__":
    run()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Single-source shortest paths: solutions (continued)

Verification code appendix: sssp_depth_checks.py

```
"""Finite SSSP checks against enumerated simple paths/cycles, not proof substitutes."""
from math import inf
from itertools import product
from random import Random
from collections import deque

SAMPLE=[(3,4,3),(1,4,8),(1,3,1),(2,4,12),(2,3,5),(2,1,-2),(0,1,4),(0,2,1)]
POSITIVE=[(u,v,2 if (u,v)==(2,1) else w) for u,v,w in SAMPLE]


def reachable(n,edges,sources):
    adj=[[] for _ in range(n)]
    for u,v,_ in edges:adj[u].append(v)
    seen=set(sources);q=deque(sources)
    while q:
        for v in adj[q.popleft()]:
            if v not in seen:seen.add(v);q.append(v)
    return seen


def oracle(n,edges,s):
    adj=[[] for _ in range(n)]
    for i,(u,v,w) in enumerate(edges):adj[u].append((v,w,i))
    d=[inf]*n;cycles=[]
    def paths(u,seen,cost):
        d[u]=min(d[u],cost)
        for v,w,_ in adj[u]:
            if v not in seen:paths(v,seen|{v},cost+w)
    paths(s,{s},0)
    for start in range(n):
        def visit(u,seen,cost,ids):
            for v,w,i in adj[u]:
                if v==start and cost+w<0:cycles.append((set(seen),ids+[i]))
                elif v not in seen:visit(v,seen|{v},cost+w,ids+[i])
        visit(start,{start},0,[])
    r=reachable(n,edges,[s]);seeds=set().union(*(vs for vs,_ in cycles if vs&r)) if cycles
else set()
    affected=reachable(n,edges,seeds)
    for v in affected:d[v]=-inf
    return d


def bellman(n,edges,s,trace=False):
    d=[inf]*n;pi=[None]*n;d[s]=0;stages=[d[:]]
    for _ in range(n-1):
        changed=False
        for i,(u,v,w) in enumerate(edges):
            if d[u]!=inf and d[u]+w<d[v]:d[v]=d[u]+w;pi[v]=i;changed=True
        stages.append(d[:])
        if not changed:break
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Single-source shortest paths: solutions (continued)

```
    bad=[i for i,(u,v,w) in enumerate(edges) if d[u]!=inf and d[u]+w<d[v]]
    return d,pi,bad,stages


def budget(n,edges,s):
    d=[inf]*n;d[s]=0;layers=[d[:]];parents=[]
    for _ in range(n):
        old=d;d=old[:];p=[None]*n
        for i,(u,v,w) in enumerate(edges):
            if old[u]!=inf and old[u]+w<d[v]:d[v]=old[u]+w;p[v]=i
        layers.append(d);parents.append(p)
    seeds={v for v in range(n) if layers[n][v]<layers[n-1][v]}
    affected=reachable(n,edges,seeds)
    result=layers[n-1][:]
    for v in affected:result[v]=-inf
    return result,layers,parents,seeds


def negative_witness(n,edges,s):
    _,layers,parents,seeds=budget(n,edges,s)
    if not seeds:return None
    v=min(seeds);back=[]
    for i in range(n,0,-1):
        identity=parents[i-1][v]
        if identity is not None:back.append(identity);v=edges[identity][0]
    walk=list(reversed(back));assert v==s and len(walk)==n
    vertices=[s];stack_edges=[];where={s:0};u=s
    for identity in walk:
        a,b,w=edges[identity];assert a==u;u=b
        if b in where:
            j=where[b];cycle=stack_edges[j:]+[identity]
            if sum(edges[k][2] for k in cycle)<0:return cycle
            for x in vertices[j+1:]:del where[x]
            vertices=vertices[:j+1];stack_edges=stack_edges[:j]
        else:
            stack_edges.append(identity);where[b]=len(vertices);vertices.append(b)
    raise AssertionError('strict nth budget improvement must contain a negative cycle')


def dense_dijkstra(n,edges,s):
    assert all(w>=0 for _,_,w in edges)
    adj=[[] for _ in range(n)]
    for i,(u,v,w) in enumerate(edges):adj[u].append((v,w,i))
    d=[inf]*n;pi=[None]*n;d[s]=0;pending=set(range(n));trace=[]
    while pending:
        u=min(pending,key=lambda v:(d[v],v));pending.remove(u)
        if d[u]==inf:break
        for v,w,i in adj[u]:
```

Single-source shortest paths: solutions (continued)

```
            if v in pending and d[u]+w<d[v]:d[v]=d[u]+w;pi[v]=i
        trace.append((u,d[u],d[:],pi[:]))
    return d,pi,trace


def dag(n,edges,s):
    indegree=[0]*n;adj=[[] for _ in range(n)]
    for i,(u,v,w) in enumerate(edges):indegree[v]+=1;adj[u].append((v,w,i))
    q=deque(v for v in range(n) if indegree[v]==0);order=[]
    while q:
        u=q.popleft();order.append(u)
        for v,w,i in adj[u]:
            indegree[v]-=1
            if indegree[v]==0:q.append(v)
    if len(order)!=n:return None
    d=[inf]*n;pi=[None]*n;d[s]=0
    for u in order:
        for v,w,i in adj[u]:
            if d[u]!=inf and d[u]+w<d[v]:d[v]=d[u]+w;pi[v]=i
    return d,pi


def certificate(n,edges,s,d,pi):
    r=reachable(n,edges,[s])
    if d[s]!=0 or pi[s] is not None:return False
    children=[[] for _ in range(n)]
    for v in range(n):
        if v not in r:
            if d[v]!=inf or pi[v] is not None:return False
        else:
            if d[v] in (inf,-inf):return False
            if v!=s:
                i=pi[v]
                if i is None or not 0<=i<len(edges):return False
                u,to,w=edges[i]
                if to!=v or d[v]!=d[u]+w:return False
                children[u].append(v)
    seen={s};q=deque([s])
    while q:
        for v in children[q.popleft()]:
            if v in seen:return False
            seen.add(v);q.append(v)
    return seen==r and all(d[v]<=d[u]+w for u,v,w in edges if u in r)


def verify(n,edges,s):
    expected=oracle(n,edges,s);actual,layers,parents,seeds=budget(n,edges,s)
    assert actual==expected
    d,pi,bad,_=bellman(n,edges,s)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Single-source shortest paths: solutions (continued)

```
    assert bool(bad)==any(v==-inf for v in expected)
    if not bad:assert d==expected and certificate(n,edges,s,d,pi)
    witness=negative_witness(n,edges,s)
    if bad:
        assert witness and sum(edges[i][2] for i in witness)<0
        assert all(edges[witness[i]][1]==edges[witness[(i+1)%len(witness)]][0] for i in
range(len(witness)))
        assert edges[witness[0]][0] in reachable(n,edges,[s])
    else:assert witness is None
    acyclic=dag(n,edges,s)
    if acyclic is not None:assert acyclic[0]==expected and certificate(n,edges,s,*acyclic)
    if all(w>=0 for _,_,w in edges):
        out=dense_dijkstra(n,edges,s);assert out[0]==expected and
certificate(n,edges,s,*out[:2])
    return bad


def main():
    rng=Random(24024);cases=sources=0
    for n in range(1,4):
        slots=[(u,v) for u in range(n) for v in range(n)]
        for values in product((None,-1,2),repeat=len(slots)):
            edges=[(u,v,w) for (u,v),w in zip(slots,values) if w is not None]
            for s in range(n):verify(n,edges,s);sources+=1
            cases+=1
    for _ in range(300):
        n=rng.randrange(4,7);edges=[(u,v,rng.randrange(-3,7)) for u in range(n) for v in
range(n) if rng.random()<.22]
        verify(n,edges,rng.randrange(n));cases+=1;sources+=1
    d,pi,bad,stages=bellman(6,SAMPLE,0)
    assert d==[0,-1,1,0,3,inf] and not bad
    assert stages==[[0,inf,inf,inf,inf,inf],[0,4,1,inf,inf,inf],[0,-1,1,5,12,inf],
[0,-1,1,0,7,inf],[0,-1,1,0,3,inf],[0,-1,1,0,3,inf]]
    pos=dense_dijkstra(6,POSITIVE,0)
    assert [(u,v) for u,v,*_ in pos[2]]==[(0,0),(2,1),(1,3),(3,4),(4,7)]
    neg=SAMPLE+[(3,1,-2)];assert budget(6,neg,0)[0]==[0,-inf,1,-inf,-inf,inf]
    zero=[(0,1,0),(1,2,0),(2,1,0)];dd,pp,_,_=bellman(3,zero,0)
    assert certificate(3,zero,0,dd,pp)
    assert not certificate(3,zero,0,dd,[None,2,1])
    constraints=[(0,1,-2),(1,2,-1),(2,0,4)]
    augmented=constraints+[(3,v,0) for v in range(3)]
    assert bellman(4,augmented,3)[0]==[0,-2,-3,0]
    assert bellman(4,[(0,1,-2),(1,2,-1),(2,0,2)]+[(3,v,0) for v in range(3)],3)[2]
    print(f'PASS weighted_graphs={cases} source_cases={sources} simple_path_cycle_oracle=True
'
          'nativeBF=budget_classification=DAG=Dijkstra_on_domains=True '
          'every_negative_witness_reachable_closed_strictly_negative=True
zero_cycle_parent_rejected=True '
          'sample_BF=0,-1,1,0,3,inf Dijkstra_order=s,b,a,c,t difference_constraints=True')


if __name__=='__main__':main()
```

Single-source shortest paths: solutions (continued)

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study